

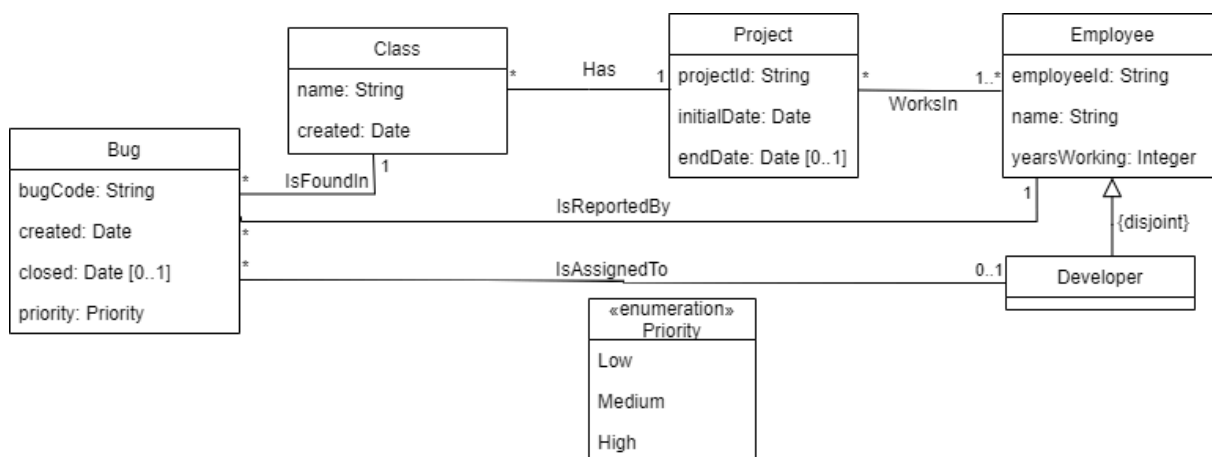
Análisis y Diseño con Patrones

Práctica 2: Patrones de Diseño y de Asignación de Responsabilidades

(evaluada sobre 50 puntos)

En la Práctica 1, una empresa nos ha pedido crear un software para gestionar los bugs que se producen en el desarrollo de proyectos de software orientados a objetos y así gestionar la asignación a desarrolladores de los bugs que se detectan en los proyectos. Ahora nos pide que añadamos nuevas funcionalidades en el software.

A continuación disponéis del diagrama estático de análisis de la Práctica 1 del que partiremos:



Claves de las clases:

- Project: projectId
- Class: projectId, name
- Bug: projectId, name, bugCode
- Employee: employeeId

Restricciones de integridad:

- La fecha final de un proyecto, si existe, debe ser posterior a la fecha de inicio del proyecto.

- La fecha de creación de una clase de un proyecto es posterior o igual a la fecha de inicio del proyecto y anterior a la fecha de finalización (si existe).
- La fecha de apertura de un bug debe ser posterior a la fecha de creación de la clase en la que se ha encontrado el bug.
- La fecha de cierre de un bug debe ser posterior a la fecha de su apertura.
- El bug de un proyecto debe ser reportado por un empleado del mismo proyecto.
- El bug de un proyecto debe ser asignado a un desarrollador del propio proyecto.

Ejercicio 1 (12,5 puntos)

Queremos implementar una nueva funcionalidad que a partir de un desarrollador, pueda obtener todos los bugs que tiene asignados y todavía no están cerrados. Llamaremos a esta funcionalidad “getAllOpenBugs”.

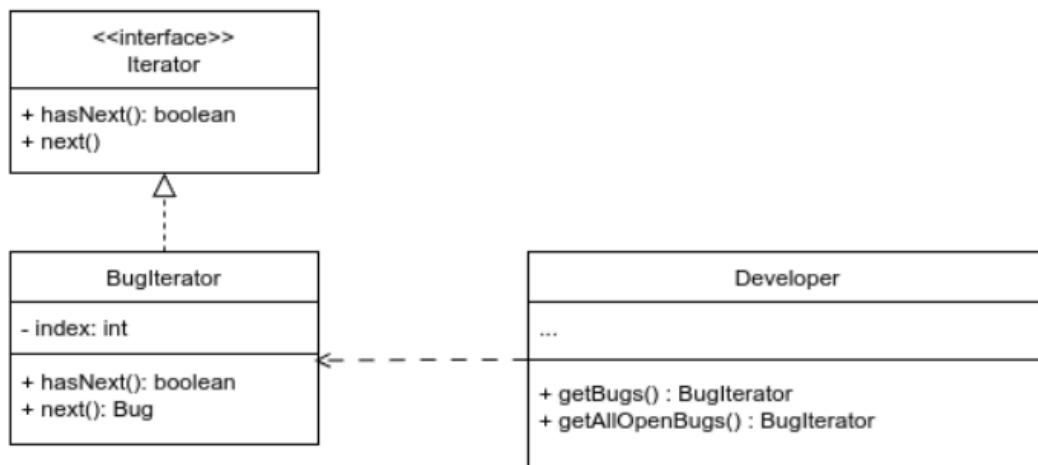
En este momento del diseño del sistema todavía no sabemos qué estructura de datos utilizaremos para guardar los bugs asignados a un desarrollador. Es esencial mantener un nivel de acoplamiento reducido durante la implementación de esta operación, como es de práctica común. Este enfoque asegurará que cualquier modificación o actualización realizada en esta funcionalidad no genere un impacto considerable en otras áreas del sistema.

Se pide:

- a) (2,5 puntos) ¿Qué patrón de diseño usarías para diseñar la nueva funcionalidad?
- b) (5 puntos) Haz el diagrama de clases con las operaciones resultantes después de aplicar los patrones.
- c) (5 puntos) Muestra el diagrama de secuencia o el pseudocódigo de la operación, así como el de las propiedades, clases y métodos necesarios para implementarlo completo.

Solución:

- a) El patrón **Iterator** se puede usar para desacoplar la lógica transversal de las clases. La clase *BugIterator* encapsula el proceso de iteración y proporciona una forma estandarizada de iterar sobre los bugs. El patrón **Iterator** se utiliza para proporcionar una forma de acceder secuencialmente a los elementos de una colección sin exponer su representación interna
- b) El diagrama de clases:



- c) El pseudocódigo:

```

class Bug {
    Date closed;

    // Other properties and methods...

    boolean isClosed() {
        return closed != null;
    }
}

class Developer {
    private List<Bug> assignedBugs;

    // Other properties and methods...
    public BugIterator getBugs() {
        return new BugIterator(this.assignedBugs);
    }
}
  
```

```
public BugIterator getAllOpenBugs() {
    List<Bug> openBugs = new List<Bug>();
    BugIterator iterator = this.getBugs();

    while (iterator.hasNext()) {
        Bug bug = iterator.next();
        if (!bug.isClosed()) {
            openBugs.add(bug);
        }
    }
    return new BugIterator(openBugs);
}

class BugIterator {
    private List<Bug> bugs;
    private int position = 0;

    public BugIterator(List<Bug> bugs) {
        this.bugs = bugs;
    }

    public boolean hasNext() {
        return this.position < this.bugs.size();
    }

    public Bug next() {
        if (this.hasNext()) {
            Bug bug = this.bugs.get(this.position);
            this.position += 1;
            return bug;
        } else {
            throw new NoSuchElementException();
        }
    }
}
```

Ejercicio 2 (12,5 puntos)

Se ha identificado que la asignación de bugs debe considerar la prioridad del bug y los años de experiencia que tiene el desarrollador en la empresa. Esta asignación dinámica puede variar según diferentes criterios que pueda establecer la empresa. En la actualidad la empresa sólo dispone de un único criterio que consiste en asignar los bugs críticos (prioridad High) a los desarrolladores con mayor experiencia (más de 5 años en la compañía), mientras que los bugs de menor prioridad (prioridad Low y Medium) se asignan a los desarrolladores menos experimentados (menos de 5 años en la compañía) para facilitar su aprendizaje y desarrollo profesional.

La asignación se hará entre una lista de desarrolladores candidatos utilizando la operación `Bug::assignBug(List<Developer> candidates)`.

En el futuro se prevé que la empresa establezca nuevos criterios de asignación de bugs según sus necesidades.

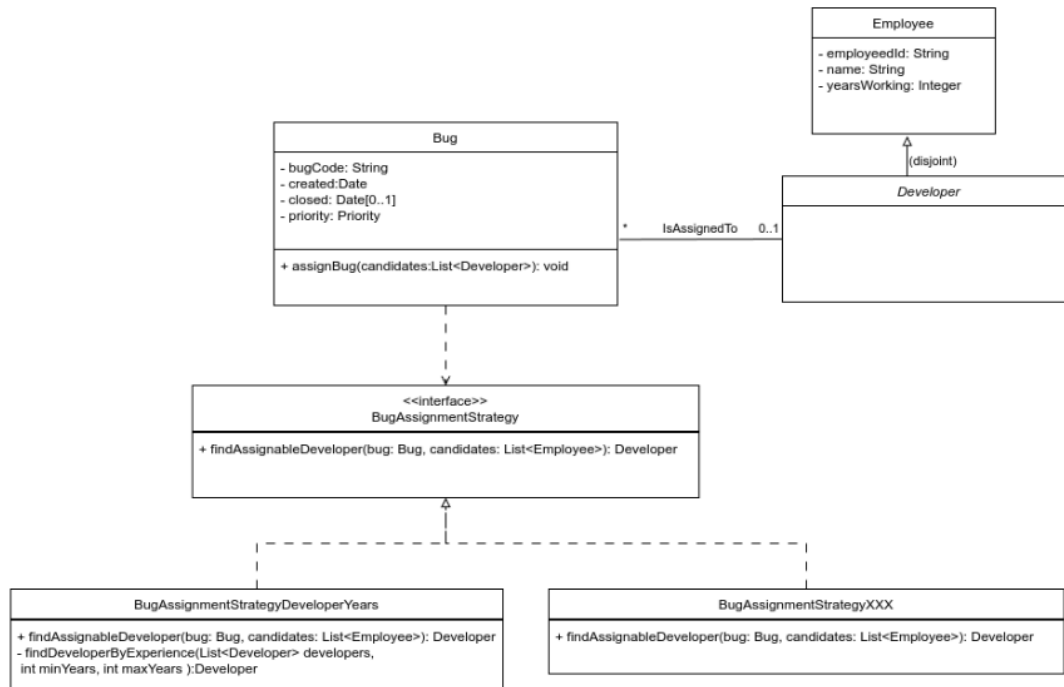
Dadas estas consideraciones, se solicita diseñar e implementar una solución que permita gestionar la asignación de bugs de manera flexible y eficiente, considerando el criterio de la prioridad del bug y los años de experiencia de los desarrolladores en la empresa. La solución que propongáis tiene que permitir añadir en el futuro nuevos criterios de asignación.

- a) (2,5 puntos) Justifica qué patrón de diseño se tiene que aplicar para resolver la situación planteada.
- b) (5 puntos) Muestra el diagrama de clases con las operaciones necesarias para implementar el diseño, añadiendo otro tipo de asignación de bugs de vuestra elección.
- c) (5 puntos) Muestra el pseudocódigo completo de las clases con las operaciones incluyendo la operación `Bug::assignBug(List<Developer> candidates)`.

Solución:

- a) El patrón **Estrategia** es ideal para este escenario. Al considerar la necesidad de asignar bugs a los desarrolladores en función de la prioridad del bug y los años de experiencia de los programadores en la empresa, surge la necesidad de una solución flexible y desacoplada. El patrón Estrategia permite encapsular diferentes algoritmos de asignación en objetos separados, lo que facilita la adaptación dinámica del comportamiento del sistema según las condiciones cambiantes.

b)



```

c) class Bug {
    private Developer assignedDeveloper;
    private BugAssignmentStrategy strategy;
    // ...

    public void assignBug(List<Developer> candidates) {
        Developer selectedDeveloper = strategy.findAssignableDeveloper(this, candidates);
        if (assignedDeveloper != null) {
            assignedDeveloper = selectedDeveloper;
        } else {
            throw new Exception("No suitable developer found for Bug " + id);
        }
    }
}

interface BugAssignmentStrategy {
    Developer findAssignableDeveloper(Bug bug, List<Developer> candidates)
}

class BugAssignmentStrategyByDeveloperYears implements BugAssignmentStrategy {
    @Override
    public Developer findAssignableDeveloper(Bug bug, List<Developer> candidates) {
        if (bug.priority == Priority.HIGH) {
            return findDeveloperByExperience(developers, 5, Integer.MAX_VALUE);
        } else {
            return findDeveloperByExperience(developers, 0, 4);
        }
    }

    private Developer findDeveloperByExperience(List<Developer> developers, int minYears,
        int maxYears) {
        foreach (Developer developer in developers) {

            if (developer.yearsOfExperience >= minYears
                && developer.yearsOfExperience <= maxYears) {
                return developer;
            }

            throw new Exception("A developer with the specified experience was not found.");
        }
    }
}

```

Ejercicio 3 (12,5 puntos)

Para agilizar el proceso de resolución de bugs para los desarrolladores, se pide un sistema flexible y modular que permita gestionar el ciclo de vida de los bugs, desde su creación hasta su resolución y cierre, considerando diferentes formas en las que se puede tener un bug como "Abierto" (hará referencia a cuando se ha creado el bug) y "Cerrado" (hará referencia a cuando se cierre el bug esté resuelto o no, ya que se podría dar el caso que no se pudiera resolver).

Habrà ciertas restricciones:

- Un bug únicamente se podrá "Cerrar" cuando esté "Abierto"
- Un bug se creará como "Abierto" y se podrá "Abrir" cuando esté "Cerrado"

Cuando un bug esté "CERRADO" se le podrá indicar una razón que puede ser "RESOLVED", "INVALID", "WONT FIX", y "CANT REPRODUCE".

También se registrará la fecha en la que se produzcan cambios en los bugs dentro de su ciclo de vida (de Abierto a Cerrado, de Cerrado a Abierto, etc...).

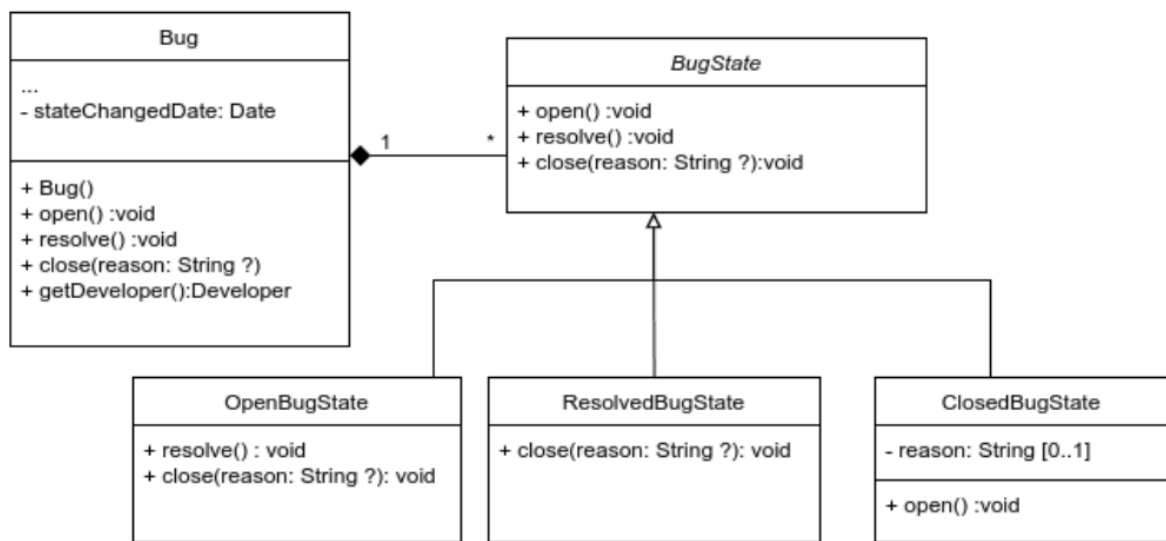
Se pide:

- a) (2,5 puntos) ¿Qué patrón de diseño utilizarías para diseñar este nuevo requisito?
- b) (5 puntos) Haz el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar solo la parte de aplicación del patrón).
- c) (5 puntos) Muestra el diagrama de secuencia o el pseudocódigo de las operaciones que permiten cambiar un bug dentro de su ciclo de vida.

Solución:

- a) El patrón **Estado** es ideal cuando un objeto cambia su comportamiento en función de su estado interno, y cuando las operaciones que realiza dependen del estado actual del objeto. En este caso, cada bug puede estar en un estado diferente (como "Abierto", "Resuelto", "Cerrado", etc.), y las operaciones que se pueden realizar en un bug (como marcar como resuelto, cerrar, etc.) dependen del estado actual del bug.

b)



c)

```
class Bug {
    private Date stateChangedDate;
    private BugState state;

    public Bug() {
        this.state = new OpenBugState();
        this.created = new Date();
        this.stateChangedDate = createdDate;
    }
    public void open() { state.open(this); }
    public void close(String reason) { state.close(this, reason); }
    public void setState(BugState state) {
        this.state = state;
    }
}
```

```

        public void setStateChangedDate(Date time) {
            this.stateChangedDate = time;
        }
        public void setClosedReason(String reason) {
            this.closedReason = reason;
        }
    }

    interface BugState {
        void open(Bug bug);
        void close(Bug bug, String reason);
    }

    class OpenBugState implements BugState {
        @Override
        public void close(Bug bug, String reason) {
            bug.setState(new ClosedBugState(reason));
            bug.setStateChangedDate(new Date());
            bug.setClosedReason(reason);
        }
    }

    // ClosedBugState class
    class ClosedBugState implements BugState {
        private String reason;
        public ClosedBugState(String reason) {      this.reason = reason;    }

        @Override
        public void open(Bug bug) {
            bug.setState(new OpenBugState());
            bug.setStateChangedDate(new Date());
            bug.setClosedReason("");
        }
    }

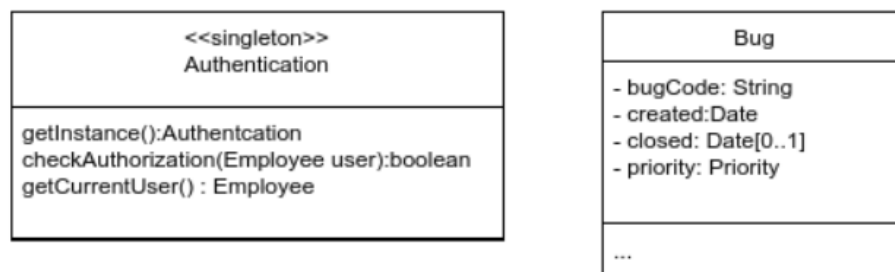
```

Ejercicio 4 (12,5 puntos)

La empresa desea aumentar la seguridad del sistema haciendo que únicamente algunos usuarios externos autorizados puedan acceder a la información de los bugs.

Se dispone una clase Singleton Authentication que gestiona la autorización de los usuarios.

La comprobación de autorización se realiza mediante el método `Authentication::checkAuthorization(user: Employee)`, el cual devuelve `true` si el usuario está autorizado y `false` en caso contrario. También se cuenta con la función `Authentication::getCurrentUser() : Employee` para obtener el usuario actual.



Con el objetivo de mejorar la seguridad del sistema, se requiere que la solución implemente un mecanismo que verifique la autorización de los usuarios antes de permitirles acceder a ciertas funciones sensibles del bug, como `Bug::getBugCode():String`, entre otras. Es esencial que esta comprobación de seguridad se realice de forma transparente para los clientes existentes del sistema, minimizando así la necesidad de modificar su código.

Esta redefinición del enunciado sugiere el uso de un componente intermediario que se encargue de gestionar el acceso a las funciones sensibles, sin que los clientes existentes tengan que preocuparse por la implementación de la seguridad.

Se lanzará la excepción `NoAuthorizationException` si un usuario no autorizado intenta acceder a una función sensible.

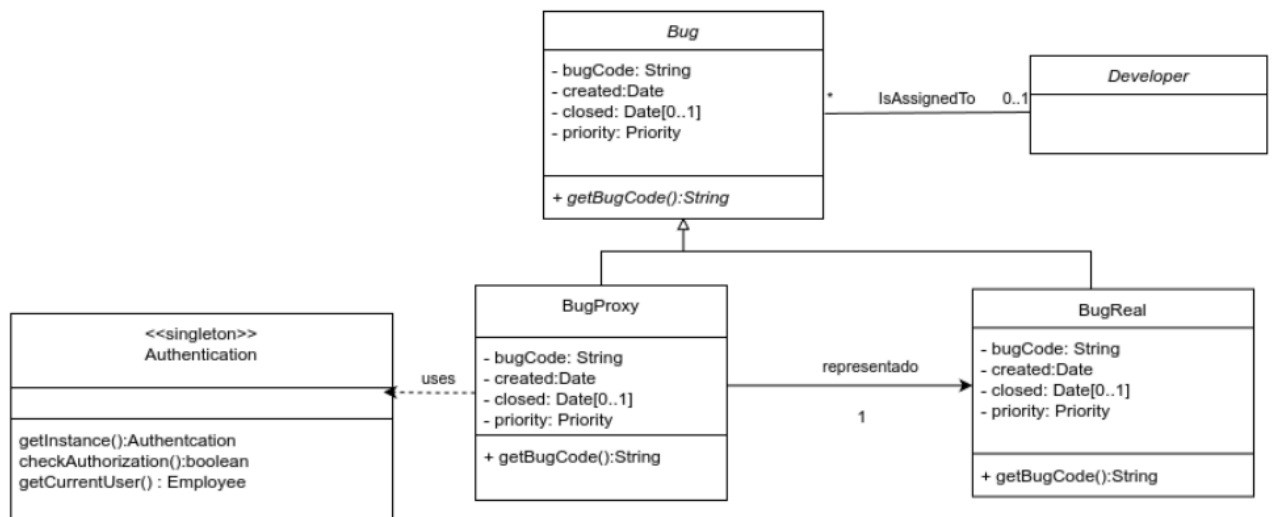
Se pide:

- (2,5 puntos) ¿Qué patrón de diseño utilizarías para diseñar este nuevo requisito de seguridad?
- (5 puntos) Haz el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar solo la parte de aplicación del patrón).
- (5 puntos) Muestra el pseudocódigo de la operación `Bug::getBugCode():String`.

Solución:

- a) El patrón de diseño más adecuado para implementar este escenario es el patrón **Representante (Proxy)** puesto a su capacidad para actuar como un intermediario entre un cliente y un objeto real, permitiendo un control refinado sobre el acceso y la manipulación de dicho objeto. Al posibilitar la implementación de lógica de validación, autorización y optimización, como la carga diferida, el caché y la gestión de red, el Proxy ofrece una solución flexible y eficiente para una amplia gama de escenarios, desde la seguridad y el rendimiento hasta la encapsulación de funcionalidades adicionales.

b)



c)

```
class NoAuthorizationException extends Exception {
    @Override
    public String getMessage() {
        return "User is not authorized to perform this action.";
    }
}

class BugProxy extends Bug {
    private Bug represented;

    public BugProxy(Bug represented) {
        this.represented = represented;
    }

    public String getBugCode( ) throws NoAuthorizationException {
        if (!Authentication.getInstance().checkAuthorization(
            Authentication.getInstance().getCurrentUser())) {
            throw new NoAuthorizationException();
        }
        return represented.getBugCode();
    }
}
```